PRADITA University

IF140303-Web Application Development

Session-12:

WebSocket and Publish-Subscribe 1

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

1. Introduction
2. LiveView Lifecycle and Architecture
3. State Management in LiveView
4. Synchronizing LiveView with Phoenix PubSub
5. Building User-Based Access Control in LiveView
6. Adding LiveView Routes in `router.ex`
7. Configuring `if-else` Components in `live.html.heex`
8. Implementing LiveView in `ThermostatLive`
9. Implementing `GenServer` in `QueueCounter`
10. Configuring `Application` to Start `Hiwi.QueueCounter`
11. Implementing the `ModifiableLive` Module for Case Management with `LiveView`
12. Interface Design for `ModifiableLive` in `modifiable_live.html.heex`
13. Implementation of `DisplayOnlyLive` for Real-Time Queue Display
14. Implementing `QueueCounterComponent` as a `LiveComponent` for Displaying Queue Numbers
15. Creating `display_only_live.html.heex` for the `DisplayOnlyLive` Interface
16. Ringkasan

Introduction

PRADITA University

- **Real-Time UI:** LiveView enables interactive user interfaces without requiring client-side JavaScript frameworks.
- **Server-Driven Architecture:** Most logic runs on the server, simplifying development and reducing front-end complexity.
- **Efficient Updates:** Only minimal diffs are sent over a WebSocket connection to update the UI.
- **BEAM Concurrency:** Elixir processes allow scalable handling of many simultaneous client sessions.
- **Ideal Use Cases:** Applications needing responsiveness, synchronized state, and simplified maintenance.

Core Concepts

PRADITA University

- **Server-Rendered HTML:** The server generates all DOM output and controls UI behavior.
- **Persistent WebSocket:** LiveView maintains a continuous connection for real-time updates.
- **Diff Tracking:** Only changed HTML fragments are sent to the client, improving efficiency.
- **Minimal JavaScript:** Most interactions require no custom JavaScript, reducing complexity.
- **Strong Synchronization:** Server-managed state ensures consistent data across users and sessions.

LiveView Lifecycle

PRADITA University

- **Mounting Phase:** `mount/3` initializes assigns, session information, and the initial socket state.
- **Rendering Phase:** `render/1` generates dynamic HTML based on the current assigns.
- **Diff Algorithm:** LiveView identifies minimal changes and sends efficient DOM updates.
- **Event-Driven Updates:** User actions trigger state changes and selective re-rendering.
- **Termination Phase:** When the connection ends, LiveView cleans resources and stops the process.

State Management & Events

PRADITA University

- **Socket Assigns:** All renderable data lives in the LiveView socket and drives the UI.
- **Persistent vs Ephemeral:** Long-lived state coexists with short-lived values for interactions.
- **assign/3:** State updates are performed with `assign/3`, ensuring UI consistency.
- **Event Types:** Clicks, form submissions, and keyboard actions are handled naturally.
- **handle_event/3:** Processes events, updates state, and triggers HTML diff rendering.

PubSub, Security, Pros & Cons

PRADITA University

- **PubSub Sync:** LiveViews can subscribe to topics for real-time shared state updates.
- **Collaborative Use:** Suitable for notifications, dashboards, and interactive multi-user tools.
- **Session Security:** Sensitive user data can be safely accessed in `mount/3`.
- **CSRF Protection:** Phoenix provides strong CSRF safeguards for secure interactions.
- **Strengths & Limits:** Simplifies development but adds server load and requires WebSocket support.

Goal: User-Based Access Control

PRADITA University

Overview

- **Context:** The `index.html.heex` template renders a queue list.
- **Requirement:** Only the queue owner may see control links.
- **File:** `lib/hello_web/controllers/queue_html/index.html.heex`
- **Approach:** Check the logged-in user against `queue.user_id`.
- **Result:** Unauthorized users cannot see `Control`, `Display`, `Edit`, `Delete`.

Key Idea

- Wrap all action links inside a conditional `if`.
- Condition uses `@conn.assigns.user` and `queue.user_id`.
- Links are rendered only when the user owns the queue.

Access-Controlled Queue Links (Code)

- Checks whether a user is logged in.
- Compares `user.id` with `queue.user_id`.
- Renders action links only for the owner.
- Protects queue operations from other users.

```
<%= if @conn.assigns.user &&
       @conn.assigns.user.id == queue.user_id do %>
  <div>
    |
    <.link href={~p"/queues/#{queue.id}/control"}> Control
         </.link>
    |
    <.link href={~p"/queues/#{queue.id}/display"}> Display
         </.link>
    |
    <.link href={~p"/queues/#{queue.id}/edit"}> Edit </.
        link>
    |
    <.link href={~p"/queues/#{queue.id}/delete"} method="
        delete"> Delete </.link>
    |
  </div>
<% end %>
```

How the Access Control Works

PRADITA University

Explanation

- **Condition:** The first line ensures `@conn.assigns.user` exists and its `id` matches `queue.user_id`.
- **Owner-Only Actions:** Only the owner can control, display, edit, or delete the queue.
- **Dynamic URLs:** Each link uses `~p"/queues/#queue.id/[action]"` to generate a URL for the current queue.
- **Security:** Prevents unauthorized users from seeing or manipulating other users' queues.
- **Reusability:** The same pattern can be applied to other secured resources.

Standard Controller Routes

PRADITA University

- **File:** `lib/hello_web/router.ex`.
- **Scope:** Routes under `"/"` use the `:browser` pipeline.
- **Controllers:** Handle classic HTTP requests and responses.
- **Queue Routes:** Support listing, creating, editing, and deleting queues.
- **About Page:** Simple static page served by `PageController`.

```
scope "/", HiwiWeb do
  pipe_through :browser

  get "/", QueueController, :index
  get "/about", PageController, :about
  get "/queues/new", QueueController, :new
  post "/queues", QueueController, :create
  get "/queues/:id/edit",
      QueueController, :edit
  put "/queues/:id",
      QueueController, :update
  delete "/queues/:id/delete",
         QueueController, :delete
end
```

Defining LiveView Routes

PRADITA University

- `live "/queues/:id/control"`: Real-time control page for a specific queue using `ModifiableLive`.
- `live "/queues/:id/display"`: Read-only, real-time display page using `DisplayOnlyLive`.
- `live "/queues/thermostat"`: Real-time temperature monitor using `ThermostatLive`.
- **Behavior:** All use WebSockets to push updates without full reloads.

```
1  scope "/", HiwiWeb do
2    pipe_through :browser
3
4    live "/queues/:id/control",
5         ModifiableLive
6
7    live "/queues/:id/display",
8         DisplayOnlyLive
9
10   live "/queues/thermostat",
11        ThermostatLive
12 end
```

Why Use LiveView Routes?

PRADITA University

- **Real-Time UI:** Pages update instantly when server state changes.
- **No Full Refresh:** Only diffs are sent over the WebSocket.
- **Interactive Screens:** Perfect for control panels and dashboards.
- **Consistent Paths:** Defined just like regular controller routes.
- **Separation of Concerns:** HTTP routing in `router.ex`, live logic in LiveView modules.

```elixir
scope "/", HiwiWeb do
  pipe_through :browser

  # Controller route
  get "/queues", QueueController, :index

  # LiveView route
  live "/queues/:id/control",
       ModifiableLive
end
```

- Use `live` routes whenever the page must stay open and continuously reflect server-side changes.

Header Authentication Logic

PRADITA University

Overview

- Conditional rendering controls navigation visibility.
- `@user_id` determines whether the user is logged in.
- Shows `Signout` when authenticated.
- Shows `Login with GitHub` when not authenticated.
- Implemented entirely in `live.html.heex`.

```
<%= if @user_id do %>
  <a href={~p"/auth/signout"}
     class="hover:text-zinc-700">
    Signout
  </a>
<% else %>
  <a href={~p"/auth/github"}
     class="hover:text-zinc-700">
    Login with GitHub
  </a>
<% end %>
```

Full Header Structure

PRADITA University

- Header includes logo, Phoenix version, and navigation links.
- Auth section conditionally shows login/logout options.
- Static links include Twitter, GitHub, and Phoenix docs.
- `@inner_content` renders the main LiveView body.
- `flash_group` is used for notifications.

```
<header class="px-4 sm:px-6 lg:px-8">
  ...
  <!-- Auth links rendered above -->
  <a href="https://twitter.com/elixirphoenix"
      >elixirphoenix</a>
  <a href="https://github.com/
      phoenixframework/phoenix">GitHub</a>
  ...
</header>
<main class="px-4 py-20 sm:px-6 lg:px-8">
  <div class="mx-auto max-w-2xl">
    <.flash_group flash={@flash} />
    <%= @inner_content %>
  </div>
</main>
```

How the `if-else` Logic Shapes the UI

- **if**: User is logged in → show `Signout`.
- **else**: User is not logged in → show GitHub login.
- Ensures correct access and smooth UX.
- Frequently used pattern for authenticated layouts.
- Can be extended to role-based navigation.

```
<%= if @user_id do %>
  <!-- Logged-in: show logout -->
  <a href={~p"/auth/signout"}>Signout</a>
<% else %>
  <!-- Guest: show GitHub login -->
  <a href={~p"/auth/github"}>
    Login with GitHub
  </a>
<% end %>
```

- Conditional rendering is essential for secure, user-specific navigation.
- LiveView templates make this pattern clean and reactive.

ThermostatLive: Module and Rendering Logic

- Implements a real-time temperature display.
- Uses `mount/3` to set initial state (30°C).
- `render/1` shows temperature + button.
- `phx-click` triggers server-side event handling.
- Demonstrates minimal but complete LiveView interaction.

```elixir
defmodule HiwiWeb.ThermostatLive do
  use HiwiWeb, :live_view

  def mount(_, _, socket) do
    {:ok, assign(socket, :temperature, 30)}
  end

  def render(assigns) do
    ~H"""Current temperature: <%= @temperature %>
        °C <button phx-click="inc_temperature"
        >+</button>"""
  end

  def handle_event("inc_temperature", _params,
      socket) do
    {:noreply, update(socket, :temperature, &(&1
        + 1))}
  end
end
```

Event Handling and Real-Time Updates

- `handle_event/3` processes `"inc_temperature"`.
- `update/3` increments temperature by 1.
- State changes trigger automatic re-render.
- No page reload required thanks to WebSocket connection.
- Ideal example of interactive server-driven UI.

```
1 <!-- thermostat_live.html.heex -->
2 Current temperature: <%= @temperature %>°C
3 <button phx-click="inc_temperature">+</button>
```

Execution Steps

- User clicks "+", LiveView receives event.
- Temperature state increases by 1.
- LiveView sends HTML diff to client.
- UI updates instantly without reload.

Takeaway Real-time UI updates with minimal client code— a core strength of Phoenix LiveView.

QueueCounter GenServer: Initialization

- **Goal:** Track number of handled cases globally.
- **Process:** `Hiwi.QueueCounter` runs as a GenServer.
- **State:** Holds the current case count as an integer.
- **Initialization:** Starts with 0 cases handled.
- **Name:** Registered as `__MODULE__` for easy access.

```elixir
defmodule Hiwi.QueueCounter do
  use GenServer

  alias Phoenix.PubSub

  # Initialize with the starting number
  def init(initial_number) do
    {:ok, initial_number}
  end

  # Start the GenServer with 0 cases handled
  def start_link(_) do
    GenServer.start_link(
      __MODULE__, 0, name: __MODULE__
    )
  end
  ...
end
```

Public API: Getting and Setting the Counter

Public API

- `get_current_number/0`: Reads the current case count.
- `set_current_number/1`: Updates the current case count.
- Uses `GenServer.call/2` for synchronous reads.
- Uses `GenServer.cast/2` for async updates.
- Hides GenServer details behind a simple interface.

```
1  defmodule Hiwi.QueueCounter do
2    use GenServer
3    ...
4    # Public API to get the current number
5    def get_current_number do
6      GenServer.call(__MODULE__, :
           get_current_number)
7    end
8
9    # Public API to set the current number
10   def set_current_number(new_number) do
11     GenServer.cast(__MODULE__, {:
           set_current_number, new_number})
12   end
13   ...
14 end
```

Handling Calls and Casts with PubSub Broadcast

Internal Callbacks

- **`handle_call/3`:** Returns current counter value to the caller.
- **`handle_cast/2`:** Updates state and broadcasts new value.
- **Broadcast Topic:** `"queue_counter:updates"`.
- **Message:** `{:number_update, new_number}`.
- Other components subscribe to receive real-time updates.

```elixir
...
def handle_call(:get_current_number, _from, current_number) do
  {:reply, current_number, current_number}
end

def handle_cast({:set_current_number, new_number}, _current_number) do
  PubSub.broadcast(Hiwi.PubSub, "queue_counter:updates", {:number_update, new_number})
  {:noreply, new_number}
end
...
```

Why Use `GenServer` + `Phoenix.PubSub`?

- **Centralized State:** One process owns the case counter.
- **Safe Concurrency:** GenServer serializes access to the state.
- **Real-Time Sync:** PubSub pushes updates to all subscribers.
- **Loose Coupling:** Producers and consumers communicate via topics.
- **Reusable Pattern:** Can be applied for other global counters or metrics.

Subscriber Example (LiveView)

```
1  ...
2  def mount(_p, _s, socket) do
3    Phoenix.PubSub.subscribe(Hiwi.PubSub,
         "queue_counter:updates")
4    {:ok, assign(socket,current_number,
         Hiwi.QueueCounter.
         get_current_number())}
5  end
6
7  def handle_info({:number_update,
       new_number}, socket) do
8    {:noreply,assign(socket, :
         current_number, new_number)}
9  end
10 ...
```

Adding `Hiwi.QueueCounter` as an Application Child

- GenServer `Hiwi.QueueCounter` must start with the application.
- Added to the supervision tree inside `application.ex`.
- Ensures the counter is globally available across all sessions.
- Initializes its internal state using `start_link/1`.
- Works together with PubSub for real-time broadcast.

```elixir
def start(_type, _args) do
  children = [
    HiwiWeb.Telemetry,
    Hiwi.Repo,
    {DNSCluster,
      query: Application.get_env(
        :hello, :dns_cluster_query
      ) || :ignore},
    {Phoenix.PubSub, name: Hiwi.PubSub},

    # Start the QueueCounter GenServer
    Hiwi.QueueCounter,

    {Finch, name: Hiwi.Finch},
    HiwiWeb.Endpoint
  ]
end
```

Why `Hiwi.QueueCounter` Must Run in the Supervision Tree

- **Global State Holder:** Tracks case counts for the whole system.
- **Persistent Process:** Runs independently for the entire application lifecycle.
- **API Access:** Other modules call `get_current_number/0` and `set_current_number/1`.
- **Real-Time Sync:** Publishes updates via `Phoenix.PubSub`.
- **Supervised Reliability:** Automatically restarted if it crashes.
- Running `QueueCounter` as a child ensures consistent, centralized, and real-time state management across the application.

Mounting and Initial State Setup

PRADITA University

mount/3 Overview

The function extracts `queue_id` from `params` and checks `user_id` in the session. If the user is signed in, it subscribes to PubSub, retrieves the current count and queue data, and assigns them to the `socket`. If the user is not signed in, it shows an error flash and redirects to `"/"`.

```elixir
def mount(params, session, socket) do
  %{"id" => queue_id} = params

  case session do
    %{"user_id" => user_id} ->
      :ok = Phoenix.PubSub.subscribe(Hiwi.PubSub, "
          queue_counter:updates")
      current_number = QueueCounter.get_current_number()
      queue = Repo.get!(Queue, queue_id)
      socket = socket
        |> assign(:current_number, current_number)
        |> assign(:queue_id, queue_id)
        |> assign(:user_id, user_id)
        |> assign(:queue, queue)
      {:ok, socket, layout: {HiwiWeb.Layouts, :live}}
    _ -> socket = socket
        |> put_flash(:error, "You haven't signed in.")
      {:ok, push_navigate(socket, to: "/")}
  end
end
```

Handling the "increment_number" Event

handle_event/3 Workflow
When the UI triggers "increment_number", the function loads the latest Queue, increments current_number via a changeset, and saves it using Repo.update/1. The updated value is reloaded, sent to QueueCounter .set_current_number/1 to update the GenServer, and assigned to the socket so the LiveView refreshes without a page reload.

```elixir
def handle_event("increment_number", _params,
    socket) do

  old_queue = Repo.get!(Queue, socket.assigns.
      queue_id)
  new_state = %{ "current_number" => old_queue.
      current_number + 1 }
  changeset = Queue.changeset(old_queue,
      new_state)
  Repo.update(changeset)

  current_queue = Repo.get!(Queue, socket.assigns
      .queue_id)
  new_number = current_queue.current_number
  QueueCounter.set_current_number(new_number)

  {:noreply, assign(socket, :current_number,
      new_number)}
end
```

Receiving PubSub Updates in LiveView

DITA
ersity

handle_info/2 and Real-Time Sync
`ModifiableLive` subscribes to the topic `"queue_counter:updates"` during `mount/3`. Whenever `Hiwi.QueueCounter` broadcasts a message such as `{:number_update, new_number}` through PubSub, `handle_info/2` receives it and updates `:current_number` in the socket. As a result, all subscribed sessions instantly display the same updated value, ensuring a synchronized and real-time case count across users.

```elixir
def handle_info( {:number_update, new_number
    }, socket) do
  {:noreply, assign(socket, :current_number,
      new_number)}
end
```

- User increments number → DB + QueueCounter updated.
- `QueueCounter` broadcasts new number.
- All `ModifiableLive` instances receive `:number_update`.
- UI updates automatically without page reload.

HTML Template for `ModifiableLive`

PRADITA University

```
<div class="flex">
  <!-- Left Area -->
  <div class="w-1/2 p-4 bg-gray-100 flex flex-col justify-start">
    <h1 class="text-4xl font-bold">Queue</h1>
    <div class="text-2xl font-semibold hover:text-zinc-700">Name: <%= @queue.name %></div>
    <div class="text-2xl font-semibold hover:text-zinc-700">Description:</div>
    <div class="text-2xl"><%= @queue.description %></div>
    <div class="text-2xl font-semibold hover:text-zinc-700">Max Number: <%= @queue.max_number %></div>
    <button class="mt-4 px-4 py-2 bg-blue-500 text-white text-2xl rounded hover:bg-blue-600" phx-click="increment_number">Next Number</button>
  </div>
  <!-- Right Area -->
  <div class="w-1/2 p-4 flex items-center justify-center bg-white">
    <div class="text-9xl font-bold"><%= @queue.prefix %><%= @current_number %></div>
  </div>
</div>
```

Explanation of Layout and Behavior

PRADITA University

- The root container uses `flex` to split the UI into two equal columns.
- The left area displays queue information (name, description, max number) and includes a `"Next Number"` button.
- The button uses `phx-click="increment_number"` to call `handle_event/3` in `ModifiableLive`, increasing the queue number in real time.
- The right area shows the current queue number in very large text (`text-9xl`), combining prefix + number for clear visibility.
- Tailwind CSS classes provide a clean and responsive layout, emphasizing information on the left and visibility of the active number on the right.
- The interface enables users to read queue details and increment the queue number instantly via LiveView, achieving real-time display without page reload.

Source Code — DisplayOnlyLive

PRADITA University

```elixir
defmodule HiwiWeb.DisplayOnlyLive do
  use HiwiWeb, :live_view
  alias Hiwi.QueueCounter
  alias Hiwi.Queue
  alias Hiwi.Repo

  def mount(params, _session, socket) do
    %{"id" => queue_id} = params
    :ok = Phoenix.PubSub.subscribe(Hiwi.PubSub, "queue_counter:updates")
    current_number = QueueCounter.get_current_number()
    queue = Repo.get!(Queue, queue_id)

    socket =socket
      |> assign(:current_number, current_number)
      |> assign(:queue_name, queue.name)
      |> assign(:queue_prefix, queue.prefix)
    {:ok, socket, layout: false}
  end
  def handle_info({:number_update, new_number}, socket) do
    {:noreply, assign(socket, :current_number, new_number)}
  end
end
```

How `mount/3` Sets Up the Display

PRADITA University

mount/3 Explanation

- Extracts `queue_id` from `params`.
- Subscribes to `"queue_counter:updates"` via PubSub.
- Retrieves current queue number from `QueueCounter`.
- Loads queue details from database using `Repo.get!/2`.
- Assigns `:current_number`, `:queue_name`, and `:queue_prefix` to socket.
- `layout: false` for fullscreen display without default layout.

Benefits

- Queue information becomes available instantly at mount.
- Subscribes once and continues receiving real-time updates.
- Ideal for big screens, signage, and queue display monitors.
- Lightweight interface focused only on queue visibility.
- Lower rendering and layout overhead for better performance.

Real-Time Sync via `handle_info/2`

PRADITA University

How It Works

- `QueueCounter` broadcasts updates through PubSub.
- `handle_info/2` receives `{:number_update, new_number}`.
- Updates `:current_number` in the LiveView socket.
- LiveView re-renders automatically without page reload.
- Every connected screen stays synchronized instantly.

Why It Matters

- Excellent for read-only queue display screens.
- No polling required — updates pushed through WebSockets.
- Works reliably for many viewers at the same time.
- Users always see the latest queue number in real time.
- Very low network and CPU overhead for high responsiveness.

QueueCounterComponent: Concept & Purpose

Component Overview

- A reusable `LiveComponent` for displaying queue numbers.
- Designed for real-time UI without modification controls.
- Uses `@queue_prefix` and `@current_number` passed from parent LiveView.
- Automatically re-renders when assigns change.
- Can be embedded across multiple LiveViews to avoid duplication.

```
1  defmodule HiwiWeb.QueueCounterComponent
       do
2    use Phoenix.LiveComponent
3
4    def render(assigns) do
5      ~H"""
6      <div>
7        <%= @queue_prefix %><%=
             @current_number %>
8      </div>
9      """
10   end
11 end
```

Render Logic and Practical Benefits

PRADITA University

How It Works

- Receives assigns from parent LiveView — no internal state.
- Displays queue number by interpolating prefix and number.
- Re-render automatically triggered when LiveView assign changes.

```
# Example usage inside a LiveView template
<.live_component
  module={HiwiWeb.QueueCounterComponent}
  id="counter"
  queue_prefix={@queue.prefix}
  current_number={@current_number}
/>
```

Advantages

- Consistent UI for queue numbers across pages.
- Eliminates repeated HTML templates.
- Efficient — no PubSub subscription or event handling needed.
- Perfect for dashboards and public display boards.

UI Code for `DisplayOnlyLive`

PRADITA University

Design Focus

- Full-screen layout optimized for public display.
- Queue name positioned at the top for quick identification.
- Queue number centered with extremely large font size.
- Read-only display — no interaction required.
- Uses `QueueCounterComponent` for real-time updates.

```
<div class="h-screen flex flex-col items-center">
  <div class="pt-8">
    <h1 class="text-8xl font-bold text-center"><%= @queue_name %></h1>
  </div>

  <div class="flex-grow flex items-center justify-center -mt-24">
    <div class="text-[16rem] font-bold text-center">
      <.live_component
        module={HiwiWeb.QueueCounterComponent}
        id="queue_counter"
        queue_prefix={@queue_prefix}
        current_number={@current_number}
      />
    </div>
  </div>
</div>
```

Real-Time Read-Only Display Logic

PRADITA University

How the Template Works

- Displays `@queue_prefix` + `@current_number`.
- Uses `QueueCounterComponent` for rendering.
- Read-only — no buttons or interaction.
- Updates instantly through PubSub + LiveView.

Visual Strategy

- Very large fonts for visibility.
- Minimal UI — focus only on queue info.
- Center layout for clear focal point.

```
<.live_component
  module={HiwiWeb.QueueCounterComponent}
  id="queue_counter"
  queue_prefix={@queue_prefix}
  current_number={@current_number}
/>
```

Real-Time Behavior

- Receives PubSub update via `handle_info/2`.
- Assigns new `:current_number` to socket.
- LiveView re-renders automatically (no refresh).

Summary: Phoenix LiveView

PRADITA University

- **Real-time rendering.** LiveView enables interactive UIs via WebSockets without client JS frameworks.
- **Efficient lifecycle.** `mount/3` initializes state, `render/1` updates only changed DOM via diff tracking.
- **Server-state model.** All UI data lives in `socket.assigns`, updated through `handle_event/3`.
- **PubSub sync.** Shared state is broadcast instantly across user sessions for consistent views.
- **Security support.** Sessions, authorization checks, and CSRF reduce risk in production apps.
- **Case example.** `ModifiableLive` + `DisplayOnlyLive` demonstrate real queue updates powered by `QueueCounter GenServer`.